Reflection Journal Report for Lab 07

Prepared for Prof. Vishwantha Rao, HCC ITAI 1371 Machine Learning
Prepared by FA26_ML_6171_16347 3_ITAI1371 Group
20-Oct-2026

The purpose of this report is for each student to reflect on their learning journey for ITAI 1371 Machine Learning (ML) Lab 07. This Reflection Journal Report consists of separate parts for each student in the group. Each student in the group was responsible for their own content and report formatting in each of the sections below.

This reflection journal entry is for Lab 07 was prepared by **Eric Roberson**.

Instead of being the first to complete this lab, I was last because I have been under the weather. I had to take 1.5 days off from work and was not able to do much schoolwork until Tuesday Oct. 20th.

Understanding Insights & New Breakthroughs

I am finally feeling like machine learning is going to be worth something. Reading through the PowerPoint gave me new insight into ML's function in the field of AI. Before this module, I thought a good model was just one with a high accuracy number. The "Accuracy Lie" slide changed that. A lazy model that predicts "Legitimate" for every transaction gets 99.9% accuracy and catches 0% of the fraud. My breakthrough was realizing that building a model is only half the job. Knowing whether it actually works is the other half, and that is what makes ML useful in AI. We have to measure what matters and that's what this what this module taught me.

Analyzing the Problem / Problem Solving Strategies

The module taught me a strategy for approaching classification problems. First, ask which mistake costs more, a false positive (a false alarm) or a false negative (a miss). That choice decides whether I care more about precision or recall. Second, compare the model to a baseline, such as always guessing the majority class. In the Titanic data, about 62% of passengers did not survive, so a model that guesses "died" for everyone already scores around 62%. Third, use cross-validation instead of trusting one train-test split. I also used problem solving on the lab itself. The code cells weren't producing output because the line breaks had collapsed and everything was being treated as a comment. I fixed it by clearing each cell and re-entering the code one piece at a time, then running the cells in order from top to bottom. My responses in the markdown cell accurately captures this:

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.

1. Describe a real-world scenario where you would care more about a model's Precision than its Recall.

We want to minimize false positives. we only want to predict yes when we are really sure. An exmaple is an email from your boss being marked as spam (postive). You could lose your job if you miss the email or meeting invite.

2. Describe a real-world scenario where you would care more about a model's Recall than its Precision.

We want to minimize the misses with Recall. We would rather have false alarms than missing a real case. In real life, an example would be cancer screening in medicine. It would be better to have a false positive than a negative which would be catastrophic for the patient if a real cancer case were missed.

3. Why is a cross-validation score generally more trustworthy than a score from a single train-test split?

A single train-test split can be lucky or unlucky. If the easy examples happen to land in the test set, the accuracy looks better than the model really is. Cross-validation fixes this by splitting the entire dataset into folds (5 folds means 20% each). The model is trained and tested 5 times, with a different fold held out as the test set each time, so every data point gets used for testing exactly once. The final score is the average across all folds, which gives a more reliable estimate of real performance. The standard deviation of the fold scores also shows how stable the model is.

Real-World Applications

The slides give clear examples. A spam filter needs high precision, because an email from your boss landing in spam is costly. Cancer screening needs high recall, because telling a sick patient they are healthy is catastrophic. Fraud detection shows why accuracy fails on imbalanced data. The fire alarm analogy (a false alarm from toast versus a silent alarm during a real fire) made the confusion matrix easy to remember. In my own work in technical risk and controls, I can see the same tradeoff in monitoring and alerting. If we flag

everything, teams get buried in noise – you boil the ocean. If we are too picky, we miss the real failure and miss what's sizzling on the back burner.

Connecting New & Prior Knowledge

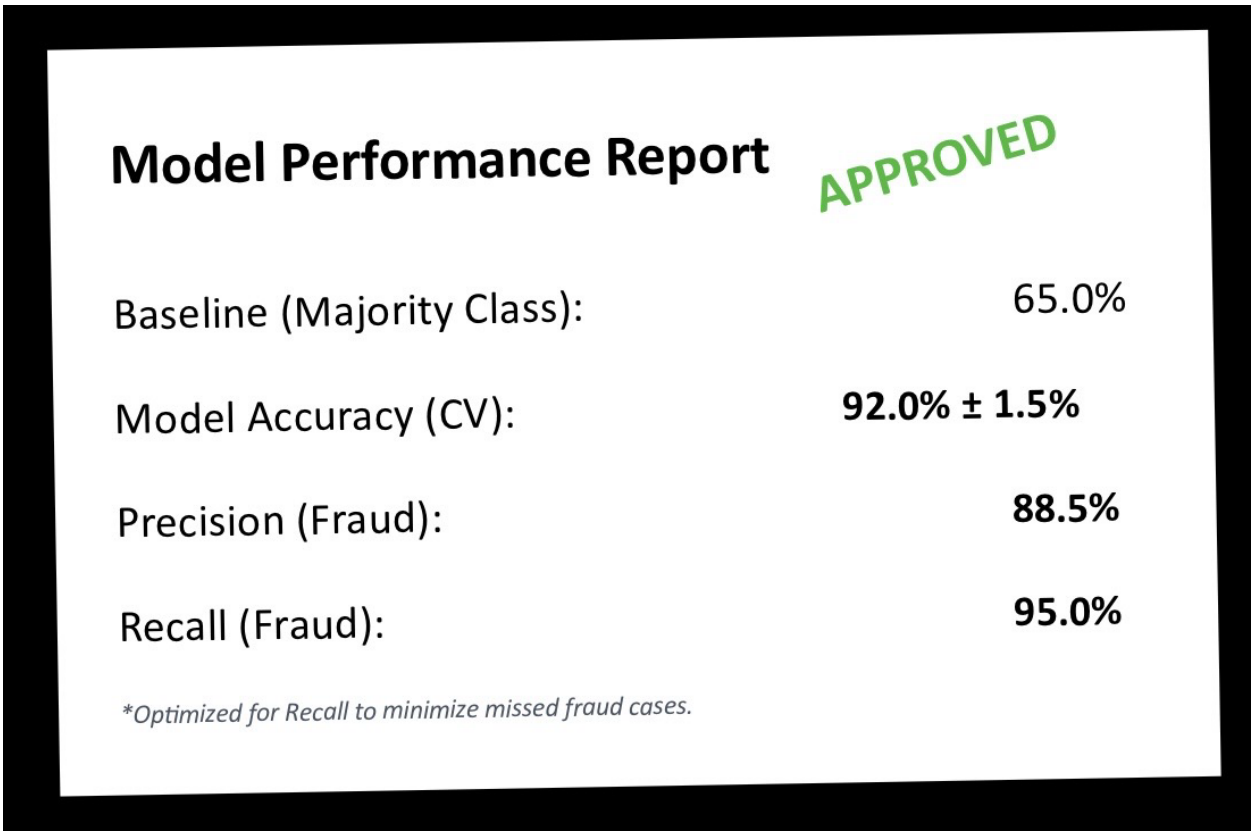
The course is building momentum and the mistakes from the midterm are starting to make more sense. I already knew how to split data into training and test sets and how to calculate accuracy, and this module showed me the limits of both. One test set can be lucky or unlucky, and accuracy hides the type of error being made. Cross-validation also connects to my work experience. Testing a control once and calling it effective is weak evidence, but if it holds up across several tests, I can trust it more. Controls don't just pass or fail; rather when testing, they are rated effective, largely effective, partially effective, or ineffective. Cross-validation applies these same ideas to AI models.

Emerging Questions

- How do people decide the right threshold in practice, and who decides how much a false positive or false negative costs when the answer isn't obvious?
- The slides covered ROC curves and AUC, but our lab didn't have us plot one. I want to how these work, when they are useful, and how they compare to the confusion matrix.
- How do data scientists validate models when cross-validation is too slow, such as with deep learning? Is 2 weeks really too slow in a risk-averse organization?
- How does this apply to our group project? For a kid-safe label, I need to think about which is worse: marking an unsafe title as safe, or blocking a safe one.

Changing Approach & Critical Thinking

I'm asking harder questions now. Before this module, we trained the model, check the accuracy, and moved on. Now, we want to know what is the baseline? Is the score stable across folds? Which error matters most for this problem? Is any information leaking from the test set or from the future? I also learned to report results the way the "Pro Way" slide describes, with a baseline, a variance (±), and a business metric like precision or recall, instead of one number like "92% accuracy." When a result looks good, I now ask "compared to what?" before I believe it.



This reflection journal entry is for Lab 07 Model Evaluation and was prepared by Irene "Alex" Ramos:

The purpose of this report is to reflect on my learning journey for ITAI 1371 Machine Learning Lab 07, which focused on moving beyond simple accuracy metrics to implement robust model evaluation techniques using the Titanic dataset.

Understanding Insights & New Breakthroughs

A primary breakthrough in this lab was recognizing the limitations of relying solely on baseline accuracy scores. Through hands-on practice, I learned how easily an accuracy score can be misleading when class distributions are uneven, and how utilizing a confusion matrix breaks down classifier performance into True Positives, True Negatives, False Positives, and False Negatives visualized through a Seaborn heatmap.

Analyzing the Problem / Problem Solving Strategies

When working through the tasks, I had to carefully configure scikit-learn metrics and build a baseline classifier using LogisticRegression to predict passenger survival based on

features like Age, Pclass, Sex, and Fare. Furthermore, implementing 5-fold cross-validation via cross_val_score provided a stable, averaged performance score (achieving an average cross-validation score of roughly 78.68% with a low standard deviation of 0.0094), which mitigated the "luck of the draw" effect from a single train-test split.

Real-World Applications

Evaluating classification models using precision, recall, and F1-score from a classification report directly translates to practical system design. In security operations and anomaly detection, tuning a model to prioritize precision (when false positives carry high costs, such as filtering legitimate traffic or emails) versus recall (when missing a positive case carries critical risk, such as failing to flag an intrusion) prevents costly operational blind spots.

Connecting New & Prior Knowledge

This lab effectively bridged foundational binary classification concepts with rigorous model validation pipelines. Using cross-validation reinforced the necessity of testing algorithms across multiple data folds to ensure models generalize reliably to unseen data rather than memorizing a single training subset.

Emerging Questions

As I analyzed the precision-recall trade-offs across different model thresholds, it raised the question: how can we dynamically adjust classification thresholds in production systems to balance business costs when the cost of false positives versus false negatives shifts over time?

Changing Approach & Critical Thinking

Moving forward, I will always incorporate cross-validation and detailed classification reports during the evaluation phase rather than stopping at standard accuracy scores. Understanding the cost asymmetry of classification errors is essential before deploying predictive models in real-world environments.

This reflection journal entry is for Lab 07 Model Evaluation and was prepared by Yuanyuan Kou:

Firstly, I learned how to evaluate a classification model using a confusion matrix, precision, recall, F1-score, and cross-validation. The model used Titanic passenger information to predict survival. This lab helped me understand why accuracy alone cannot tell us everything about a model.

The confusion matrix made the prediction results easier to understand. The model correctly predicted 90 passengers who died and 54 passengers who survived. It also predicted survival for 15 passengers who died and predicted death for 20 passengers who survived. Looking at these numbers helped me see the two different types of mistakes.

The main challenge for me is remembering the difference between precision and recall. Both use true positives, so their formulas can look similar. I can understand them better by asking two questions. Precision asks, “Of the passengers predicted to survive, how many actually survived?” Recall asks, “Of all the passengers who actually survived, how many did the model find?” For survival, the model had about 78% precision and 73% recall. This means it still missed some actual survivors.

I also learned why cross-validation is useful. A single train-test split may give a different score depending on how the data is divided. Five-fold cross-validation evaluates the model using five different validation groups. The average accuracy was about 78.68%. Checking several folds gives me a better picture of how consistently the model performs.

This lesson connects to our Netflix movie project. We want to predict whether a movie may be appropriate for children. If we define unsafe movies as the positive class, recall would be especially important because we do not want to miss movies that need review. This also helped me understand that I must check what the positive class means before interpreting precision and recall.

After this lab, I understand that choosing an evaluation metric depends on the goal of the project. I still need more practice reading classification reports and explaining the results in my own words. In future work, I will look at the types of mistakes a model makes and use several metrics before deciding whether it performs well.

This reflection journal entry is for Lab 07 Regression and Classification was prepared by Carmen Esono

In this lab, I learned how to evaluate a machine learning model using different techniques, including the confusion matrix, precision, recall, F1-score, and cross-validation. I used the Titanic dataset to predict whether passengers survived or died. This activity helped me understand how machine learning models make predictions and how we can measure their performance.

One of the most important things I learned is that accuracy alone is not enough to determine whether a model is good. The confusion matrix shows the correct and incorrect predictions, helping us understand the mistakes made by the model. Precision tells us how many positive

predictions were correct, while recall measures how many actual positive cases the model identified. The F1-score combines precision and recall into one measurement.

I also learned about cross-validation, which tests a model using different parts of the dataset. This method is useful because it gives us a more reliable idea of how the model might perform with new data. Instead of depending on only one train-test split, we can compare the results from several tests.

Another important lesson was understanding how these metrics apply to real-world situations. For example, precision is important in email spam filtering because we do not want important messages to be incorrectly marked as spam. Recall is especially important in cybersecurity because missing a real threat could cause serious problems.

Overall, this lab helped me improve my understanding of machine learning and model evaluation. I learned that choosing the right evaluation metrics depends on the problem we are trying to solve. In the future, I will pay more attention to different performance measurements instead of looking only at accuracy. These skills will help me build and evaluate more reliable machine learning models.

This reflection journal entry is for Lab 07 Regression and Classification and was prepared by Xiomara Torres:

This lab has helped me understand the difference between a model's overall accuracy and the types of mistakes it makes. Our model correctly predicted 144 out of 179 passenger outcomes, which gives it an 80.4% accuracy score. The confusion matrix also showed 15

false positives and 20 false negatives. Those results helped me understand why we can't rely on one accuracy score to explain everything about a model's performance.

I started off preparing the data by converting the ‘Sex’ column into numerical values and filling missing ages with the median. I used Age, Pclass, Sex and Fare to train the logistic regression model after splitting the dataset. I corrected formatting problems and addressed a pandas warning by assigning the filled ages directly back to the age column. Working through these issues helped me better understand the code. The classification report showed approximately 78% precision.

I was able to connect my Python practice with a complete machine learning workflow. I learned that importing libraries provides tools, while model.fit() learns patterns from training examples and model.predict() applies those patterns to other inputs. I also learned that the logistic regression model we used does not contain artificial neurons which clarified a misconception I had about machine learning.

During this lab I questioned what would happen if a dataset or library became unavailable. This led me to think about saving trained models and backing up data.

Moving forward, I will look beyond overall accuracy and examine individual errors and variation across evaluations. I also want to practice modifying and explaining the code independently so I can build confidence in both programming and model evaluation.